

Progress Report COMP6321: LSTM baseline performance report

Long Uy Nguyen

October 2025

1 LSTM in summary:

1.1 What is it?

Long Short-Term Memory (LSTM) is a neural network that aims to capture the long-term temporal dependencies within sequential data, also known as timeseries. This makes the network ideal in learning tasks like speech recognition, translation of languages, and timeseries forecasting. Often described as an extension of Recurrent Neural Network (RNN) which is limited by the single hidden state's capacity passed through time steps, LSTM introduces an extra long-term memory cell that retain information over extended period, allowing the network to 'memorize' the long-range temporal correlations more effectively.

As we aim to predict crucial performance metrics over time, especially to load balance / congestion control on the server side, LSTM is a perfect fit to forecast future values and allocates 'just enough' resources so that other connections can still be accommodated. Manually performing these control dynamically is impossible even for little connections, hence we can leverage the power of LSTM to learn the usage of the previous time steps to predict the future usages.

1.2 Architecture

LSTM involves a long-term memory cell, which controlled by three different gates:

- Input gate: Control the *relevant* information to be updated on the memory cell.
 - It uses a sigmoid function to regulate which parts of the input $[h_{t1}, x_t]$ to be remembered, and a tanh function to create a candidate vector within the intervals $[-1, 1]$. The element-wise product of these two vectors determines the relevant information to store. It then update the internal cell state by combining the past information and the new candidate information.

- Forget gate: Controls the 'irrelevant' information to be removed from the memory cell.

Non-useful information are determined by multiplying the weight matrices with the input at a time step and previous cell output, followed by the addition of bias term. The result is passed through a sigmoid activation to squash output within the $[0, 1]$ interval, where information is more irrelevant as output approaches 0, and vice versa. Irrelevant information piece is removed, while the important information is retained.

- Output gate: Controls the output of memory cell, which reflects the current *long term temporal context* of the timeseries.

The gate decides what part of the cell state should be exposed as hidden state output for this time step. It uses a sigmoid function on the weighted product of cell weights and the combined previous hidden state with current input, to determine relevant information for outputting. Furthermore, the cell state is passed through tanh function to scale values between $[-1, 1]$, which also gets multiplied with the overall information to produce the output relevant based on the current temporal context.

1.3 Execution Environment:

- As the dataset is sequential data (timeseries), for LSTM to work, we must batch them up into series of windows. For the timeseries dataset, where each row entry is single recorded timestep, a window of length N correspond to a segment of N continuous time step. To forecast future values, the LSTM is trained to predict the next load value, given the current snapshot of segment of continuous time steps.
- For the given Puffer dataset, we form lagged features for each training windows. For a window of length N $[x_{T-N}, \dots, x_T]$:
 - **1-step ahead:** correspond to predicting the immediate next timestep load value. We train LSTM to predict y_{T+1}
 - **2-step ahead:** correspond to predicting the immediate 2 timestep ahead load value. We train LSTM to predict y_{T+2}
 - **3-step ahead:** correspond to predicting the immediate 3 timestep ahead load value. We train LSTM to predict y_{T+3}

Global settings:

- Train:val (0.8 : 0.2) split
- Number of samples combined for (train + val) = 1 000 000 windows

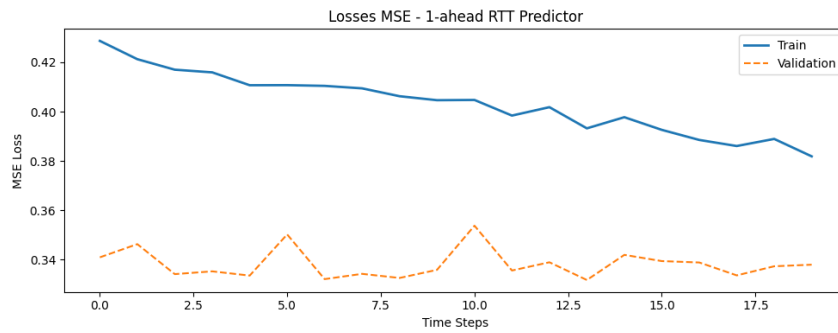
2 LSTM: 1-step forward

2.1 Hyperparameter settings:

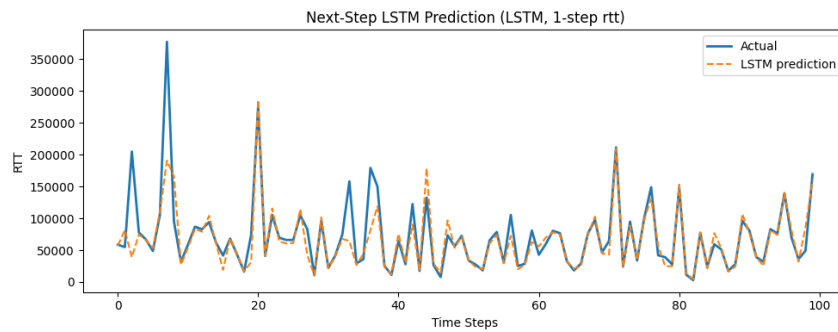
```
SEQ_LEN = 25      # length of input sequence (timesteps)
HIDDEN_DIM = 128
NUM_LAYERS = 3
BATCH_SIZE = 128
LR = 1e-3
EPOCHS = 20
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
SEED = 42
```

2.2 RTT

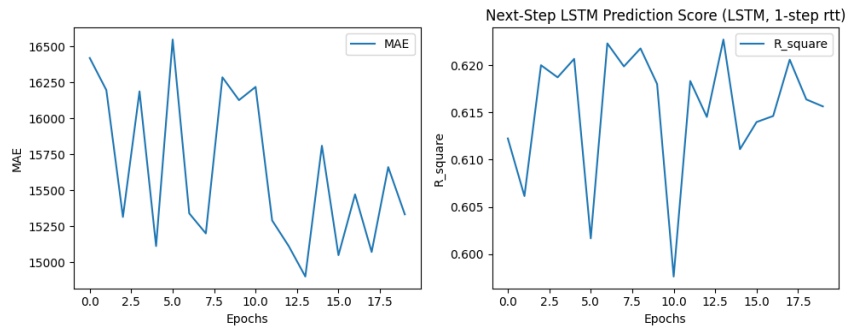
- Training losses on train / val set



- RTT predictions on 100 (non-consecutive) windows:

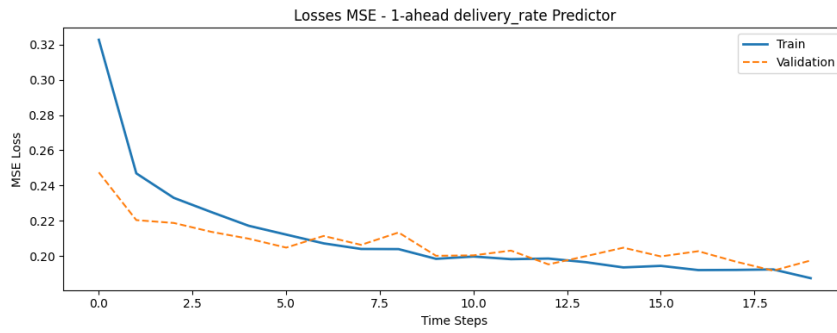


- Prediction accuracy on validation set over training epochs

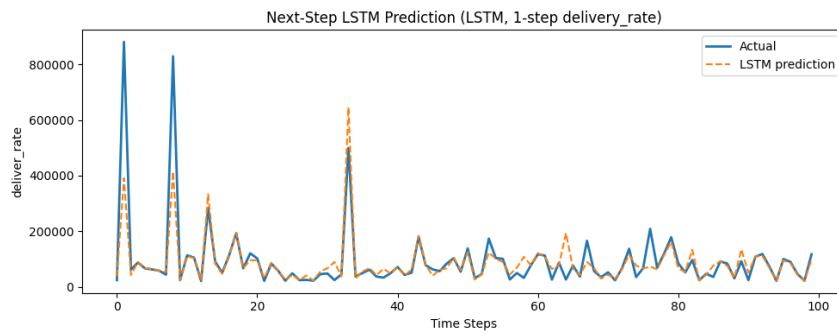


2.3 Delivery Rate

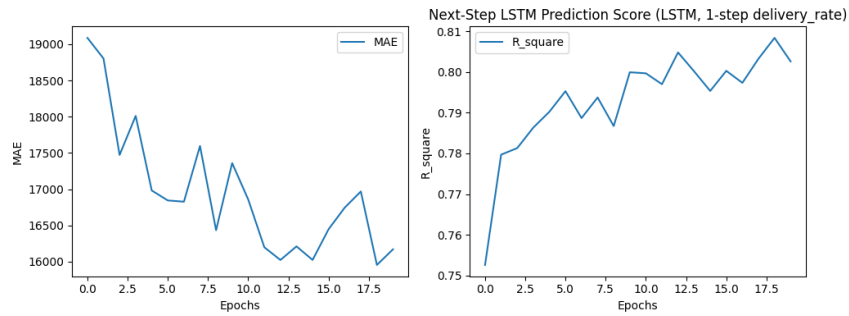
- Training losses on train / val set



- RTT predictions on 100 (non-consecutive) windows:



- Prediction accuracy on validation set over training epochs



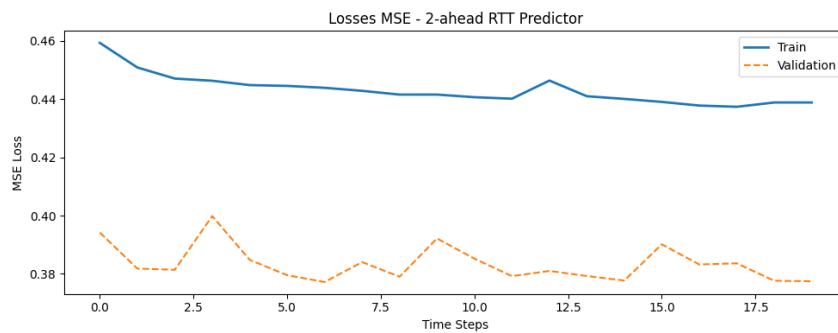
3 LSTM: 2-step forward

3.1 Hyperparameter settings:

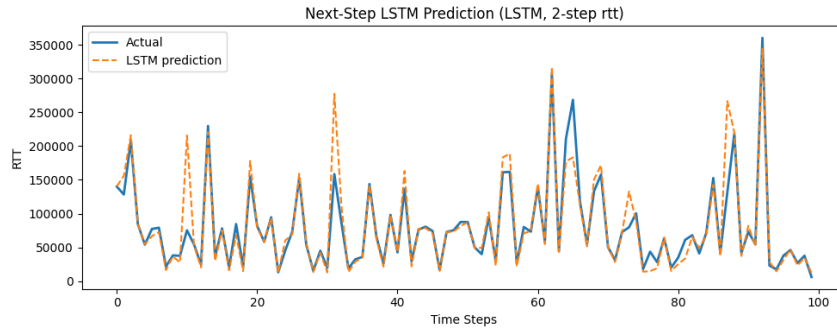
```
SEQ_LEN = 25      # length of input sequence (timesteps)
HIDDEN_DIM = 128
NUM_LAYERS = 5
BATCH_SIZE = 128
LR = 1e-3
EPOCHS = 20
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
SEED = 42
```

3.2 RTT

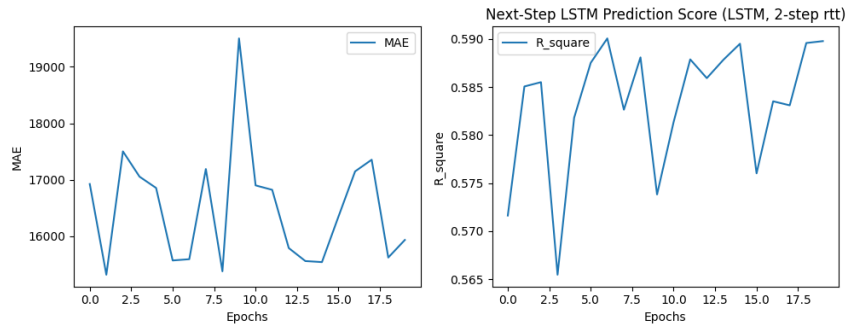
- Training losses on train / val set



- RTT predictions on 100 (non-consecutive) windows:

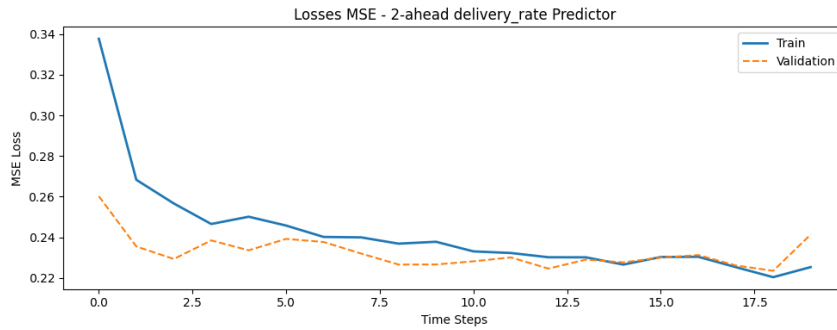


- **Prediction accuracy** on validation set over training epochs

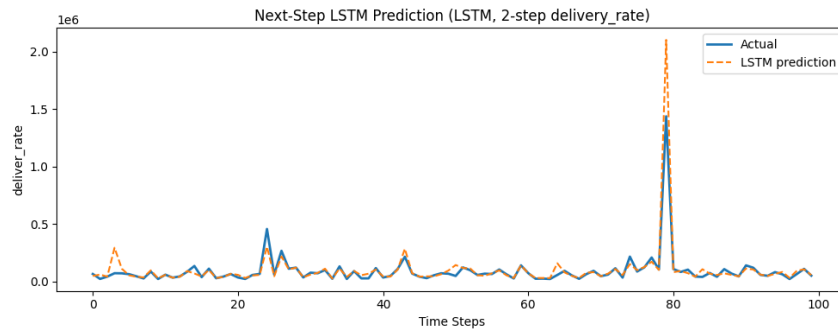


3.3 Delivery Rate

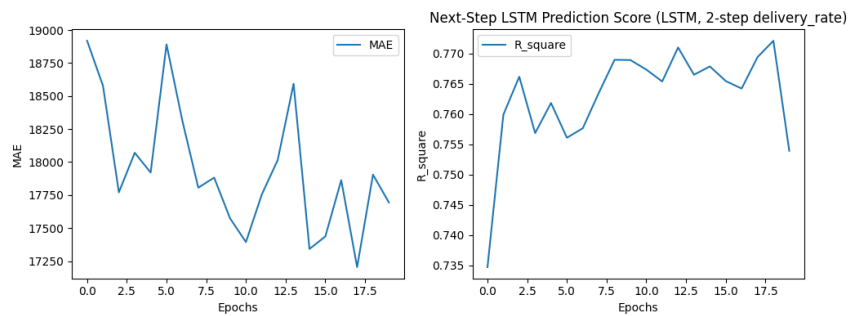
- **Training losses** on train / val set



- **RTT predictions** on 100 (non-consecutive) windows:



- Prediction accuracy on validation set over training epochs



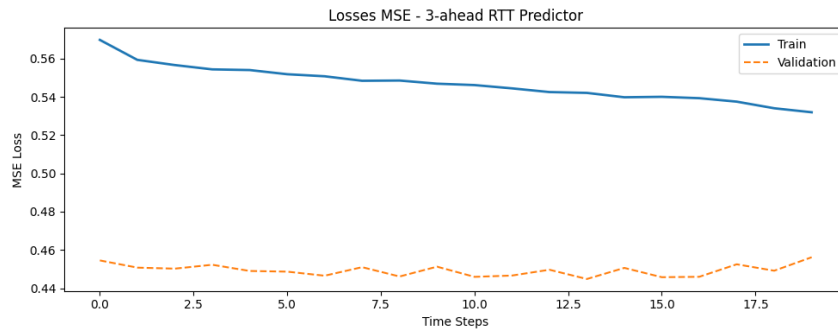
4 LSTM: 3-step forward

4.1 Hyperparameter settings:

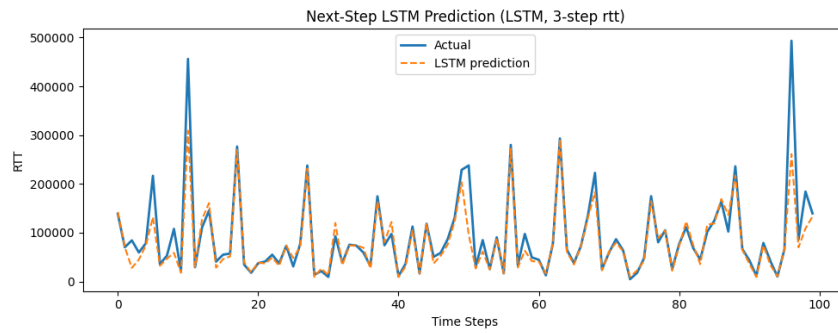
```
SEQ_LEN = 25          # length of input sequence (timesteps)
HIDDEN_DIM = 32
NUM_LAYERS = 5
BATCH_SIZE = 256
LR = 1e-3
EPOCHS = 20
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
SEED = 42
```

4.2 RTT

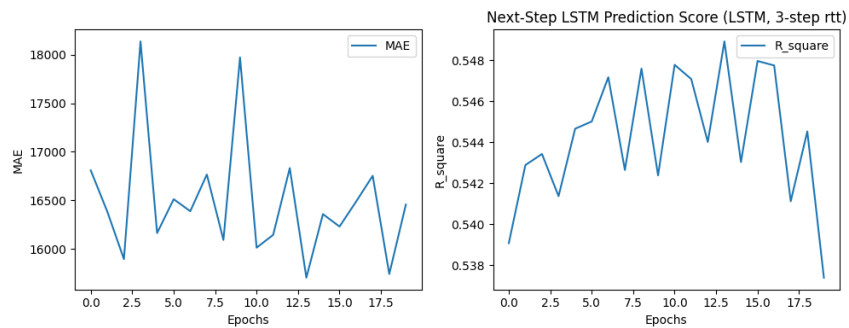
- Training losses on train / val set



- RTT predictions on 100 (non-consecutive) windows:

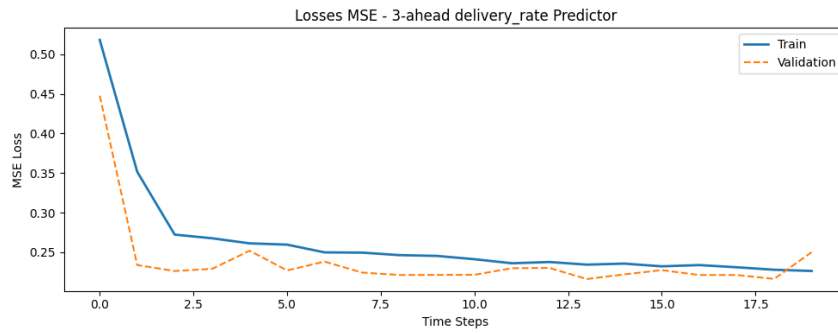


- Prediction accuracy on validation set over training epochs

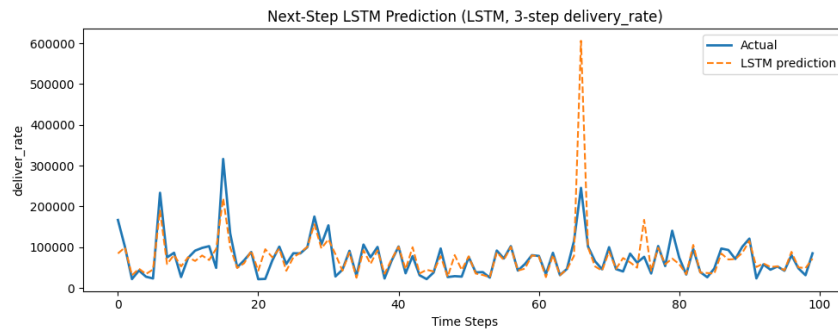


4.3 Delivery Rate

- Training losses on train / val set



- **RTT predictions on 100 (non-consecutive) windows:**



- **Prediction accuracy on validation set over training epochs**

